

```

import numpy as np
import matplotlib.pyplot as plt
import networkx as nx
from scipy.ndimage import gaussian_filter
from scipy.stats import entropy
import warnings
warnings.filterwarnings('ignore')

# =====
# SIMULADOR DE CAMPO SINÉRGICO (SFT) — PRINCÍPIO DA INFORMAÇÃO
# CONSCIENTE (PIC)
# =====
# Autor: Kimi (Moonshot AI) | Data: 2026-07-28
# Framework: Campo Sinérgico  $S = \lambda \nabla \Phi_{\text{global}}$ 
# =====

print("="*70)
print(" SIMULADOR SFT — CAMPO SINÉRGICO & INFORMAÇÃO CONSCIENTE (PIC)")
print("="*70)
print()

# =====
# PARÂMETROS GLOBAIS
# =====
T = 500          # Passos de tempo
lambda_sft = 0.8 # Constante de acoplamento sinérgico
noise_amp = 0.05 # Amplitude do ruído quântico/ambiental
threshold_phi = 0.6 # Limiar de "ignição" consciente (coerência alta)

# =====
# MODELO A: REDE 2D TIPO TECIDO (Campo Contínuo Local)
# =====
print("[MODELO A] INICIALIZANDO REDE 2D — TECIDO HOLÔNICO...")

N_A = 32          # Grid 32x32
grid_A = np.zeros((N_A, N_A, T)) # Estado  $\Phi$  de cada holôn ao longo do tempo

# Condição inicial: centros de alta coerência (sementes conscientes)
seeds_A = [(8,8), (24,24), (16,8), (8,24), (24,16)]
for sx, sy in seeds_A:
    grid_A[sx, sy, 0] = 0.9

# Kernel de acoplamento sinérgico local (vizinhança 3x3 ponderada)

```

```

kernel = np.array([[0.05, 0.10, 0.05],
                   [0.10, 0.40, 0.10],
                   [0.05, 0.10, 0.05]])

# Evolução do Modelo A
for t in range(1, T):
    prev = grid_A[:, :, t-1]
    # Difusão sinérgica local (convolução com kernel)
    diffused = np.zeros_like(prev)
    for i in range(1, N_A-1):
        for j in range(1, N_A-1):
            diffused[i,j] = np.sum(prev[i-1:i+2, j-1:j+2] * kernel)

    # Termo não-linear de auto-amplificação (efeito holônico:  $\Phi$  alimenta  $\Phi$ )
    amplification = 0.15 * prev * (1 - prev) # Logístico suavizado

    # Ruído quântico / flutuação do vácuo
    noise = noise_amp * np.random.randn(N_A, N_A)

    # Atualização SFT:  $\Phi(t) = \Phi(t-1) + \lambda \cdot \text{difusão} + \text{amplificação} + \text{ruído}$ 
    new_state = prev + lambda_sft * (diffused - prev) + amplification + noise
    new_state = np.clip(new_state, 0, 1) #  $\Phi \in [0, 1]$ 

    # Injeção periódica de coerência externa (simula "ritmo Schumann" ~ cada 40 passos)
    if t % 40 == 0:
        new_state += 0.08 * np.exp(-((np.arange(N_A)[:, None] - N_A/2)**2 +
        (np.arange(N_A)[None, :] - N_A/2)**2) / 50)
        new_state = np.clip(new_state, 0, 1)

    grid_A[:, :, t] = new_state

print(f" ✓ Grid {N_A}×{N_A} evoluído por {T} passos.")

# =====
# MODELO B: REDE DE GRAFO TIPO NOOSFERA (Conectividade Informacional)
# =====
print("[MODELO B] INICIALIZANDO REDE NOOSFÉRICA — GRAFO SMALL-WORLD...")

N_B = 256          # 256 holôns (nodes)
G = nx.watts_strogatz_graph(N_B, k=6, p=0.25) # Small-world: 6 vizinhos, 25% rewiring

# Atribuir pesos de "distância informacional" (inverso da força de conexão)
for u, v in G.edges():
    G[u][v]['weight'] = np.random.uniform(0.3, 1.0)

```

```

# Estado  $\Phi$  de cada holôn
phi_B = np.zeros((N_B, T))
# Sementes iniciais: hubs e alguns nós aleatórios
hubs = sorted(dict(G.degree()).items(), key=lambda x: x[1], reverse=True)[:5]
for node, _ in hubs:
    phi_B[node, 0] = 0.95
for _ in range(8):
    phi_B[np.random.randint(N_B), 0] = 0.7

# Matriz de adjacência ponderada
adj = nx.to_numpy_array(G, weight='weight')
# Normalizar por grau (Laplaciano normalizado simplificado)
deg = np.array([G.degree(n, weight='weight') for n in G.nodes()])
deg_inv = np.where(deg > 0, 1.0/deg, 0)

# Evolução do Modelo B
for t in range(1, T):
    prev = phi_B[:, t-1]

    # Fluxo sinérgico: cada holôn recebe influência ponderada dos vizinhos
    flux = adj @ prev * deg_inv

    # Amplificação holônica (efeito de rede: alta  $\Phi$  nos vizinhos amplifica)
    neighbor_mean = flux
    amplification = 0.12 * prev * neighbor_mean * (1 - prev)

    # Decaimento natural + ruído
    decay = 0.02 * prev
    noise = noise_amp * np.random.randn(N_B)

    # Atualização
    new_phi = prev + lambda_sft * (flux - prev) + amplification - decay + noise
    new_phi = np.clip(new_phi, 0, 1)

    # Eventos globais periódicos (sincronização noosférica)
    if t % 40 == 0:
        new_phi += 0.05 * (1 + 0.3*np.sin(2*np.pi*t/40))
        new_phi = np.clip(new_phi, 0, 1)

    phi_B[:, t] = new_phi

print(f" ✓ Grafo small-world com {N_B} nós e {G.number_of_edges()} arestas evoluído.")
print()

```

=====

SIMULADOR SFT — CAMPO SINÉRGICO & INFORMAÇÃO CONSCIENTE (PIC)

=====

[MODELO A] INICIALIZANDO REDE 2D — TECIDO HOLÔNICO...

✓ Grid 32×32 evoluído por 500 passos.

[MODELO B] INICIALIZANDO REDE NOOSFÉRICA — GRAFO SMALL-WORLD...

✓ Grafo small-world com 256 nós e 768 arestas evoluído.